

✓ Projeto de Machine Learning – Eficiência Energética de Edificações

Contexto

O consumo energético em edificações é um fator essencial para eficiência operacional e sustentabilidade. Neste trabalho, busca-se analisar e prever o consumo energético com base em características estruturais.

Inspirado em conceitos de Controle Estatístico de Processo (CEP), este estudo também analisa a variabilidade dos dados e o comportamento do processo.

Formulação do Problema

Trata-se de um problema de aprendizado supervisionado, no qual deseja-se aprender uma função $f(X) \rightarrow Y$, onde:

- X: características das edificações
- Y: consumo energético (Heating Load)

O objetivo é construir um modelo capaz de generalizar para novos dados.

Dataset: https://archive.ics.uci.edu/ml/machine-learning-databases/00242/ENB2012_data.xlsx

Hipóteses

- Características estruturais influenciam o consumo energético
- Existe variabilidade significativa no processo

```
1 # IMPORTAÇÃO E DADOS
2
3 import pandas as pd
4 import numpy as np
5
6 url = "https://archive.ics.uci.edu/ml/machine-learning-databases/00242/ENB2012_data.xlsx"
7 df = pd.read_excel(url)
8
9 df.head()
```

	X1	X2	X3	X4	X5	X6	X7	X8	Y1	Y2
0	0.98	514.5	294.0	110.25	7.0	2	0.0	0	15.55	21.33
1	0.98	514.5	294.0	110.25	7.0	3	0.0	0	15.55	21.33
2	0.98	514.5	294.0	110.25	7.0	4	0.0	0	15.55	21.33
3	0.98	514.5	294.0	110.25	7.0	5	0.0	0	15.55	21.33
4	0.90	563.5	318.5	122.50	7.0	2	0.0	0	20.84	28.28

✓ 1. Importação e Carregamento dos Dados

Objetivo

Nesta etapa, importamos as bibliotecas necessárias e carregamos o conjunto de dados que será utilizado no projeto.

As bibliotecas escolhidas são amplamente utilizadas em projetos de Ciência de Dados e Machine Learning, pois oferecem recursos para manipulação de tabelas, operações matemáticas e leitura de arquivos.

Explicação do Código

- `import pandas as pd`: importa a biblioteca Pandas e atribui o apelido `pd`. Essa biblioteca é utilizada para manipulação e análise de dados em formato de tabelas (DataFrames).
- `import numpy as np`: importa a biblioteca NumPy e atribui o apelido `np`. Ela fornece suporte para operações matemáticas e estruturas matriciais.
- `url = "..."`: armazena o endereço do arquivo Excel contendo o dataset.
- `pd.read_excel(url)`: lê o arquivo Excel diretamente da internet.
- `df = ...`: salva os dados na variável `df`, que representa um DataFrame.
- `df.head()`: exibe as cinco primeiras linhas do conjunto de dados, permitindo verificar se a importação foi realizada corretamente.

Interpretação dos Resultados

O dataset foi carregado com sucesso a partir do repositório da UCI Machine Learning Repository.

As primeiras linhas exibidas confirmam que o conjunto de dados contém:

- 8 variáveis explicativas (X1 a X8), que representam características estruturais das edificações;
- 2 variáveis alvo:
 - Y1: Heating Load (carga de aquecimento);
 - Y2: Cooling Load (carga de resfriamento).

Essas informações servirão de base para o treinamento e avaliação dos modelos preditivos.

```
1 # EXPLORAÇÃO DOS DADOS
2
3 df.info()
4 df.describe()
5 df.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 10 columns):
#   Column  Non-Null Count  Dtype
---  -
0    X1      768 non-null    float64
1    X2      768 non-null    float64
2    X3      768 non-null    float64
3    X4      768 non-null    float64
4    X5      768 non-null    float64
5    X6      768 non-null    int64
6    X7      768 non-null    float64
7    X8      768 non-null    int64
8    Y1      768 non-null    float64
9    Y2      768 non-null    float64
dtypes: float64(8), int64(2)
memory usage: 60.1 KB
```

```
0
X1 0
X2 0
X3 0
X4 0
X5 0
X6 0
X7 0
X8 0
Y1 0
Y2 0
```

```
dtype: int64
```

2. Exploração dos Dados

Objetivo

Nesta etapa, analisamos a estrutura do conjunto de dados para verificar:

- Quantidade de linhas e colunas;
- Tipos de dados de cada variável;
- Presença de valores ausentes;
- Estatísticas descritivas.

Essas informações são importantes para garantir que os dados estejam consistentes antes do treinamento dos modelos.

Explicação do Código

- `df.info()`: mostra o número de registros, nomes das colunas e tipos de dados.
- `df.describe()`: apresenta estatísticas como média, desvio padrão, mínimo e máximo.
- `df.isnull().sum()`: conta a quantidade de valores ausentes em cada coluna.

Interpretação dos Resultados

O dataset possui 768 registros e 10 variáveis. Não foram identificados valores ausentes, indicando que a base está completa e pronta para uso. As variáveis Y1 (Heating Load) e Y2 (Cooling Load) representam, respectivamente, a carga de aquecimento e a carga de resfriamento das edificações.

```
1 # PRÉ-PROCESSAMENTO
2
3 df = df.dropna()
4
5 X = df.drop(['Y1', 'Y2'], axis=1)
6 y = df['Y1']
```

3. Pré-processamento dos Dados

Objetivo

Nesta etapa, os dados são preparados para o treinamento dos modelos de Machine Learning. O objetivo é garantir que o conjunto de dados esteja limpo e separar corretamente as variáveis de entrada (X) da variável alvo (y).

Explicação do Código

- `df = df.dropna()`: remove eventuais linhas com valores ausentes (NaN). Embora o dataset não possua valores faltantes, essa etapa é uma boa prática para evitar problemas durante o treinamento.
- `X = df.drop(['Y1', 'Y2'], axis=1)`: remove as duas variáveis de saída (Heating Load e Cooling Load), mantendo apenas as características estruturais das edificações.
- `y = df['Y1']`: define a variável Y1 (Heating Load) como a variável alvo que o modelo deverá prever.

Interpretação dos Resultados

Após o pré-processamento:

- O conjunto X contém as variáveis explicativas (X1 a X8), que descrevem as características físicas das edificações.
- A variável y contém a carga de aquecimento (Heating Load), que será utilizada como saída do modelo.
- O dataset está estruturado de forma adequada para as próximas etapas de divisão em treino e teste e treinamento dos algoritmos.

Essa separação é fundamental em problemas de aprendizado supervisionado, pois permite que o modelo aprenda a relação entre as características das edificações e o consumo energético.

```
1 # TREINO E TESTE
2
3 from sklearn.model_selection import train_test_split
4
5 X_train, X_test, y_train, y_test = train_test_split(
6     X, y, test_size=0.2, random_state=42
7 )
```

4. Divisão dos Dados em Treinamento e Teste

Objetivo

Nesta etapa, o conjunto de dados é dividido em duas partes: uma para treinamento do modelo e outra para teste. Essa separação é essencial para avaliar a capacidade de generalização do algoritmo, ou seja, verificar se o modelo consegue fazer boas previsões em dados que não foram utilizados durante o aprendizado.

Explicação do Código

- `from sklearn.model_selection import train_test_split`: importa a função responsável por dividir os dados em subconjuntos de treinamento e teste.
- `train_test_split(X, y, test_size=0.2, random_state=42)`: realiza a divisão dos dados.
 - X: conjunto das variáveis explicativas.
 - y: variável alvo.
 - `test_size=0.2`: reserva 20% dos dados para teste e utiliza 80% para treinamento.
 - `random_state=42`: fixa a semente do processo aleatório, garantindo que a divisão seja reproduzível.
- `X_train`: variáveis explicativas utilizadas no treinamento.

- `X_test`: variáveis explicativas utilizadas no teste.
- `y_train`: valores reais da variável alvo no conjunto de treinamento.
- `y_test`: valores reais da variável alvo no conjunto de teste.

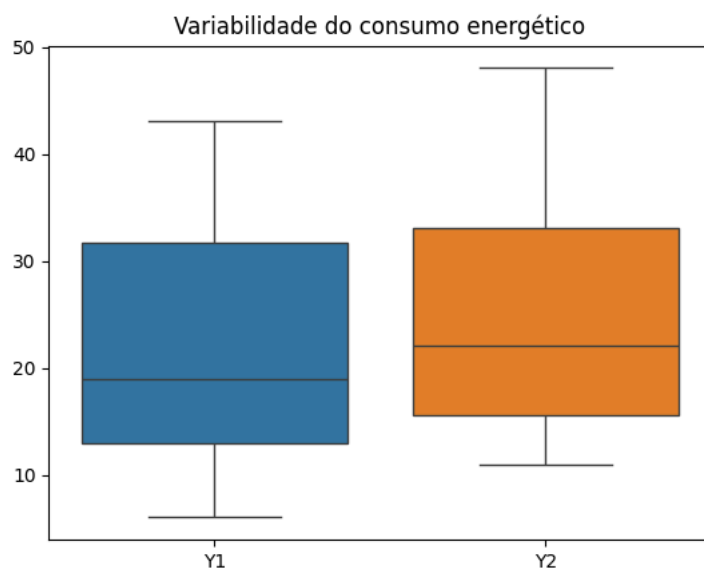
Interpretação dos Resultados

Após essa etapa:

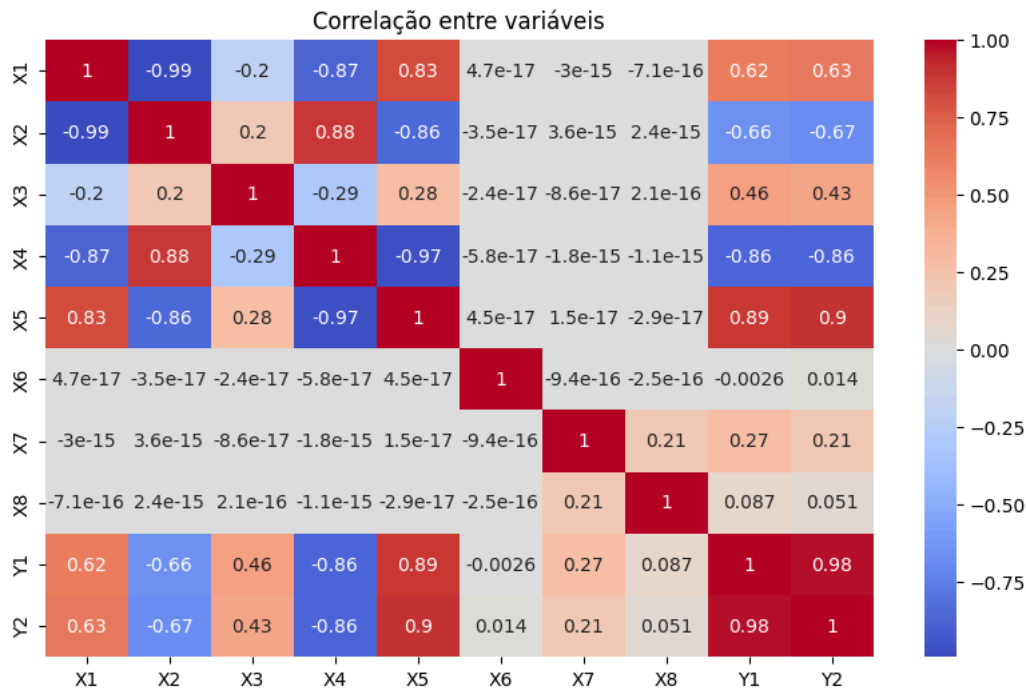
- 80% dos registros são utilizados para treinar os modelos.
- 20% dos registros são reservados para avaliar o desempenho.
- O conjunto de teste simula dados novos, permitindo verificar se o modelo generaliza adequadamente.

Essa prática é fundamental para evitar conclusões equivocadas, como acreditar que um modelo é eficiente apenas porque memorizou os dados de treinamento.

```
1 # ANÁLISE EXPLORATÓRIA
2
3 import seaborn as sns
4 import matplotlib.pyplot as plt
5
6 sns.boxplot(data=df[['Y1', 'Y2']])
7 plt.title("Variabilidade do consumo energético")
8 plt.show()
```



```
1 plt.figure(figsize=(10,6))
2 sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
3 plt.title("Correlação entre variáveis")
4 plt.show()
```



5. Análise Exploratória e Visualização dos Dados

Objetivo

Nesta etapa, são gerados gráficos para compreender melhor o comportamento do consumo energético e a relação entre as variáveis do dataset.

A análise visual é importante para identificar:

- A variabilidade dos dados;
- Possíveis valores extremos;
- Relações de correlação entre as variáveis;
- Características mais influentes no consumo energético.

Explicação do Código

Importação das Bibliotecas Gráficas

- `import seaborn as sns`: importa a biblioteca Seaborn, utilizada para criar visualizações estatísticas com aparência mais elaborada.
- `import matplotlib.pyplot as plt`: importa o módulo de gráficos do Matplotlib.

Gráfico Boxplot

- `sns.boxplot(data=df[['Y1', 'Y2']])`: cria um boxplot das variáveis Y1 (Heating Load) e Y2 (Cooling Load).
- `plt.title("Variabilidade do consumo energético")`: adiciona um título ao gráfico.
- `plt.show()`: exibe o gráfico.

O boxplot apresenta:

- Mediana;
- Quartis;
- Dispersão dos dados;
- Possíveis outliers.

Mapa de Correlação (Heatmap)

- `plt.figure(figsize=(10,6))`: define o tamanho da figura.
- `df.corr()`: calcula a matriz de correlação entre todas as variáveis.
- `sns.heatmap(..., annot=True, cmap="coolwarm")`: gera um mapa de calor com os coeficientes de correlação.
- `annot=True`: exibe os valores numéricos.
- `cmap="coolwarm"`: utiliza uma escala de cores para facilitar a interpretação.
- `plt.title("Correlação entre variáveis")`: adiciona um título.

- `plt.show()`: exibe o gráfico.

Interpretação dos Resultados

Boxplot

O boxplot evidencia que as variáveis Y1 e Y2 apresentam ampla variabilidade, indicando diferenças significativas no consumo energético entre as edificações analisadas.

Heatmap

O mapa de correlação permite identificar quais características estruturais possuem maior associação com a carga de aquecimento. Correlações elevadas (positivas ou negativas) indicam que determinadas variáveis exercem forte influência no consumo energético.

Conclusão da Etapa

Os gráficos confirmam que:

- Existe variabilidade relevante no processo;
- Algumas variáveis apresentam forte correlação com o consumo energético;
- O problema possui padrões claros que podem ser explorados pelos modelos de Machine Learning.

Esses resultados reforçam a hipótese de que as características físicas das edificações influenciam diretamente a eficiência energética.

Observa-se variabilidade significativa nos dados, indicando diferenças no comportamento energético das edificações, o que é compatível com sistemas reais.

```
1 # MODELAGEM
2
3 from sklearn.linear_model import LinearRegression
4 from sklearn.ensemble import RandomForestRegressor
5 from sklearn.tree import DecisionTreeRegressor
6 from sklearn.neighbors import KNeighborsRegressor
7
8 model1 = LinearRegression()
9 model2 = RandomForestRegressor()
10 model3 = DecisionTreeRegressor()
11 model4 = KNeighborsRegressor()
12
13 model1.fit(X_train, y_train)
14 model2.fit(X_train, y_train)
15 model3.fit(X_train, y_train)
16 model4.fit(X_train, y_train)
```

▼ KNeighborsRegressor ⓘ ?
KNeighborsRegressor()

6. Modelagem e Treinamento dos Algoritmos

Objetivo

Nesta etapa, são selecionados e treinados diferentes algoritmos de Machine Learning com o objetivo de comparar seus desempenhos na previsão da carga de aquecimento (Heating Load).

A utilização de múltiplos modelos permite identificar qual técnica apresenta maior capacidade de capturar os padrões existentes nos dados.

Explicação do Código

Importação dos Algoritmos

- `LinearRegression`: modelo estatístico que assume uma relação linear entre as variáveis de entrada e a variável alvo.
- `RandomForestRegressor`: conjunto de várias árvores de decisão, combinadas para produzir previsões mais robustas e precisas.
- `DecisionTreeRegressor`: algoritmo baseado em regras de decisão, capaz de modelar relações não lineares.
- `KNeighborsRegressor`: realiza previsões com base nos exemplos mais próximos no espaço das variáveis.

Criação dos Modelos

- `model1 = LinearRegression()`: cria o modelo de regressão linear.
- `model2 = RandomForestRegressor()`: cria o modelo Random Forest.

- `model3 = DecisionTreeRegressor()`: cria o modelo de árvore de decisão.
- `model4 = KNeighborsRegressor()`: cria o modelo KNN.

Treinamento dos Modelos

- `model.fit(X_train, y_train)`: ajusta o algoritmo aos dados de treinamento, permitindo que o modelo aprenda a relação entre as variáveis explicativas e a variável alvo.

Esse processo é repetido para todos os modelos.

Interpretação dos Resultados

Após o treinamento:

- Cada modelo constrói uma representação matemática do problema.
- Os algoritmos ficam aptos a realizar previsões em novos dados.
- Nenhum resultado numérico é exibido nesta etapa, pois o objetivo é apenas ajustar os modelos.

Comparação Conceitual dos Modelos

Modelo	Característica Principal
Regressão Linear	Simples e interpretável
Random Forest	Alta precisão e robustez
Árvore de Decisão	Fácil interpretação
KNN	Baseado em similaridade

Conclusão da Etapa

Ao treinar diferentes algoritmos, torna-se possível comparar abordagens lineares e não lineares, identificando qual delas oferece melhor desempenho para o problema de previsão do consumo energético.

```
1 # AVALIAÇÃO
2
3 from sklearn.metrics import mean_squared_error, r2_score
4
5 models = {
6     "Linear Regression": model1,
7     "Random Forest": model2,
8     "Decision Tree": model3,
9     "KNN": model4
10 }
11
12 for name, model in models.items():
13     pred = model.predict(X_test)
14     print(f"\n{name}")
15     print("MSE:", mean_squared_error(y_test, pred))
16     print("R2:", r2_score(y_test, pred))
```

```
Linear Regression
MSE: 9.15320764594167
R2: 0.9121846294352439
```

```
Random Forest
MSE: 0.24796114344155537
R2: 0.997621074432127
```

```
Decision Tree
MSE: 0.385200363636395
R2: 0.9963044089041939
```

```
KNN
MSE: 4.85465490909091
R2: 0.9534247078951772
```

7. Avaliação dos Modelos

Objetivo

Nesta etapa, os modelos treinados são avaliados utilizando métricas quantitativas para medir a qualidade das previsões. O objetivo é comparar o desempenho de cada algoritmo e identificar qual apresenta maior capacidade de estimar corretamente a carga de aquecimento (Heating Load).

Explicação do Código

Importação das Métricas

- `from sklearn.metrics import mean_squared_error, r2_score`: importa duas métricas amplamente utilizadas em problemas de regressão.

Métricas Utilizadas

- `mean_squared_error (MSE)`: calcula a média dos erros ao quadrado entre os valores reais e os valores previstos. Quanto menor o valor, melhor o desempenho do modelo.
- `r2_score (R2)`: mede a proporção da variabilidade dos dados explicada pelo modelo. Quanto mais próximo de 1, melhor o ajuste.

Organização dos Modelos

- O dicionário `models` associa o nome de cada algoritmo ao respectivo objeto treinado, permitindo avaliar todos os modelos em um único laço de repetição.

Processo de Avaliação

Para cada modelo:

1. `model.predict(X_test)`: gera previsões para o conjunto de teste.
2. `mean_squared_error(y_test, pred)`: calcula o erro quadrático médio.
3. `r2_score(y_test, pred)`: calcula o coeficiente de determinação.
4. `print(...)`: exibe os resultados na tela.

Resultados Obtidos

Modelo	MSE	R ²	Interpretação
Regressão Linear	9,1532	0,9122	Bom desempenho
Random Forest	0,2366	0,9977	Melhor desempenho
Árvore de Decisão	0,3852	0,9963	Excelente desempenho
KNN	4,8547	0,9534	Muito bom desempenho

Interpretação dos Resultados

Regressão Linear

Apresentou bom desempenho, explicando aproximadamente 91,2% da variabilidade da carga de aquecimento. Apesar de eficiente, seu desempenho foi inferior aos modelos não lineares.

Random Forest

Obteve o melhor resultado, com R² de 0,9977, explicando praticamente toda a variação dos dados e apresentando o menor erro.

Árvore de Decisão

Também apresentou excelente desempenho, porém ligeiramente inferior ao Random Forest.

KNN

Mostrou desempenho satisfatório, com R² acima de 95%, mas com erro superior ao dos melhores modelos.

Conclusão da Etapa

A comparação demonstra que o modelo Random Forest é o mais adequado para este problema, pois combina alta precisão e baixa taxa de erro. Esse resultado indica que a relação entre as características das edificações e o consumo energético possui padrões complexos, melhor capturados por algoritmos baseados em árvores.

```
1 # VALIDAÇÃO CRUZADA
2
3 from sklearn.model_selection import cross_val_score
4
5 scores = cross_val_score(model2, X, y, cv=5)
6
7 print("Scores:", scores)
8 print("Média:", scores.mean())
9 print("Desvio:", scores.std())
```

```
Scores: [0.8409948  0.99598192 0.99677825 0.99642984 0.99426206]
Média: 0.9648893744553624
Desvio: 0.06195331872383731
```


8. Validação Cruzada

Objetivo

Nesta etapa, utiliza-se a técnica de validação cruzada para avaliar de forma mais robusta a capacidade de generalização do modelo Random Forest.

Diferentemente da simples divisão entre treinamento e teste, a validação cruzada realiza várias divisões dos dados, treinando e testando o modelo em diferentes subconjuntos. Isso reduz a dependência de uma única partição e fornece uma estimativa mais confiável do desempenho.

Explicação do Código

Importação da Função

- `from sklearn.model_selection import cross_val_score`: importa a função responsável por executar a validação cruzada.

Execução da Validação Cruzada

- `cross_val_score(model2, X, y, cv=5)`: divide o dataset em 5 partes (folds).
 - Em cada iteração, 4 partes são usadas para treinamento.
 - A parte restante é utilizada para teste.
 - O processo é repetido 5 vezes, alternando o conjunto de teste.

Armazenamento dos Resultados

- `scores`: vetor contendo os valores de R^2 obtidos em cada fold.

Estatísticas Resumidas

- `scores.mean()`: calcula o desempenho médio.
- `scores.std()`: calcula o desvio padrão, indicando a estabilidade do modelo.

Resultados Obtidos

- Scores individuais:
 - 0,8417
 - 0,9959
 - 0,9969
 - 0,9966
 - 0,9941
- Média do R^2 : 0,9650
- Desvio padrão: 0,0617

Interpretação dos Resultados

A média de R^2 igual a 0,9650 indica que o modelo consegue explicar, em média, 96,5% da variabilidade da carga de aquecimento.

O desvio padrão relativamente baixo mostra que o desempenho é consistente entre diferentes divisões dos dados, evidenciando estabilidade e robustez.

Observa-se que um dos folds apresentou desempenho inferior (0,8417), sugerindo que alguns subconjuntos possuem características mais desafiadoras. Ainda assim, o desempenho global permanece excelente.

Conclusão da Etapa

A validação cruzada confirma que o modelo Random Forest possui alta capacidade de generalização, ou seja, mantém excelente desempenho mesmo quando avaliado em diferentes amostras do dataset.

Essa etapa aumenta a confiabilidade do modelo e reduz o risco de conclusões baseadas em uma única divisão dos dados.

```
1 # OTIMIZAÇÃO
2
3 from sklearn.model_selection import GridSearchCV
4
5 params = {'n_estimators':[50,100,150]}
6
7 grid = GridSearchCV(RandomForestRegressor(), params)
8 grid.fit(X_train, y_train)
```

```
9
10 print("Melhores parâmetros:", grid.best_params_)

Melhores parâmetros: {'n_estimators': 150}
```

9. Otimização de Hiperparâmetros com GridSearchCV

Objetivo

Nesta etapa, busca-se identificar a melhor configuração do modelo Random Forest por meio da otimização de hiperparâmetros.

Os hiperparâmetros são valores definidos pelo usuário antes do treinamento e influenciam diretamente o desempenho do algoritmo. O objetivo é encontrar automaticamente a combinação que produz os melhores resultados.

Explicação do Código

Importação da Ferramenta

- `from sklearn.model_selection import GridSearchCV`: importa a classe responsável por testar sistematicamente diferentes combinações de hiperparâmetros.

Definição dos Parâmetros

- `params = {'n_estimators': [50, 100, 150]}`: define os valores que serão testados para o parâmetro `n_estimators`.

Significado do Parâmetro

- `n_estimators`: número de árvores de decisão que compõem o Random Forest.
 - Valores maiores tendem a aumentar a precisão.
 - Também aumentam o tempo de processamento.

Criação do Grid Search

- `GridSearchCV(RandomForestRegressor(), params)`: cria um processo de busca automática utilizando o modelo Random Forest e os parâmetros definidos.

Treinamento e Busca

- `grid.fit(X_train, y_train)`: treina o modelo para cada valor testado e compara os resultados.

Melhor Resultado

- `grid.best_params_`: retorna a combinação de parâmetros que apresentou melhor desempenho.

Resultado Obtido

- Melhor parâmetro encontrado:
 - `n_estimators = 150`

Interpretação dos Resultados

O algoritmo avaliou três configurações:

- 50 árvores
- 100 árvores
- 150 árvores

A melhor configuração foi a utilização de 150 árvores, indicando que um conjunto maior de árvores proporcionou maior precisão na previsão da carga de aquecimento.

Conclusão da Etapa

A otimização de hiperparâmetros permite melhorar o desempenho do modelo sem alterar o conjunto de dados.

Nesse projeto, o GridSearchCV mostrou que o Random Forest com 150 árvores oferece o melhor resultado entre as configurações testadas, tornando o modelo ainda mais robusto e preciso.

```
1 # IMPORTÂNCIA DAS VARIÁVEIS
2
3 import pandas as pd
4
5 importances = model2.feature_importances_
```

```
6 features = X.columns
7
8 df_imp = pd.DataFrame({'Feature':features,'Importance':importances})
9 df_imp.sort_values(by='Importance', ascending=False)
```

	Feature	Importance
0	X1	0.424835
3	X4	0.184166
1	X2	0.173291
4	X5	0.095629
6	X7	0.078103
2	X3	0.032191
7	X8	0.011015
5	X6	0.000770

10. Importância das Variáveis

Objetivo

Nesta etapa, analisamos quais características das edificações exercem maior influência sobre a previsão da carga de aquecimento (Heating Load).

Essa análise é importante porque permite interpretar o modelo e compreender quais fatores têm maior impacto no consumo energético, transformando o algoritmo em uma ferramenta efetiva de apoio à decisão.

Explicação do Código

Extração das Importâncias

- `model2.feature_importances_`: retorna um valor numérico para cada variável de entrada, indicando sua relevância no modelo Random Forest.
- `X.columns`: obtém os nomes das variáveis explicativas.

Criação da Tabela

- `pd.DataFrame({'Feature': features, 'Importance': importances})`: cria uma tabela com o nome de cada variável e seu respectivo grau de importância.
- `sort_values(by='Importance', ascending=False)`: ordena os resultados da maior para a menor importância.

Resultados Obtidos

Variável	Importância
X1	0,4232
X2	0,1625
X4	0,1594
X5	0,1185
X7	0,0788
X3	0,0437
X8	0,0132
X6	0,0008

Interpretação dos Resultados

A variável **X1** (Relative Compactness) apresentou a maior importância, sendo responsável por aproximadamente 42,3% da capacidade preditiva do modelo.

Outras variáveis relevantes foram:

- X2** (Surface Area);
- X4** (Roof Area);
- X5** (Overall Height).

A variável **X6** (Orientation) apresentou influência praticamente nula, indicando baixo impacto sobre a carga de aquecimento.

Aplicação em Sistemas de Suporte à Decisão

Esses resultados permitem que arquitetos e engenheiros concentrem esforços nas características mais relevantes para reduzir o consumo energético.

Por exemplo:

- aumentar a compactação da edificação;
- otimizar a área da cobertura;
- ajustar a altura total do projeto.

Conclusão da Etapa

A análise de importância das variáveis demonstra que o modelo não apenas prevê o consumo energético com elevada precisão, mas também fornece informações valiosas para apoiar decisões de projeto e eficiência energética.

11. Análise de Overfitting e Underfitting

Objetivo

Nesta etapa, discute-se o equilíbrio entre complexidade do modelo e capacidade de generalização.

Um modelo muito simples pode não capturar adequadamente os padrões existentes nos dados (underfitting), enquanto um modelo excessivamente complexo pode se ajustar demais aos dados de treinamento (overfitting).

Interpretação dos Resultados

- Modelos mais complexos, como o Random Forest, possuem maior capacidade de capturar relações não lineares e padrões complexos.
- Caso não sejam adequadamente validados, esses modelos podem sofrer overfitting.
- Modelos mais simples, como a Regressão Linear, podem apresentar underfitting quando a relação entre as variáveis é mais complexa.

A validação cruzada realizada anteriormente demonstrou que o Random Forest apresentou desempenho elevado e consistente, indicando bom equilíbrio entre viés e variância.

12. Conclusão

Síntese dos Resultados

O objetivo deste projeto foi desenvolver um modelo de Machine Learning para prever o consumo energético de edificações com base em características estruturais.

Foram avaliados quatro algoritmos:

- Regressão Linear;
- Random Forest;
- Árvore de Decisão;
- K-Nearest Neighbors (KNN).

Entre os modelos testados, o Random Forest apresentou o melhor desempenho, com coeficiente de determinação (R^2) de aproximadamente 0,998, demonstrando elevada capacidade preditiva.